What Is DuckDB and Why Should I Use It?

A Modern Analytical Database That Fits in Your Pocket

Table of Contents

1. What Is DuckDB?
2. "SQLite for Analytics"
3. The Problem DuckDB Solves
4. Core Architecture: Why It's Fast
5. When Should You Use DuckDB?
6. DuckDB vs. the Alternatives
7. Superpower: Query Anything with SQL
8. Hands-On Examples
    - Your First Table
    - Filter and Sort
    - Aggregates and Grouping

What Is DuckDB?

DuckDB is a database that answers questions about data — **fast**.

It is different from databases like MySQL or PostgreSQL. Those need a separate server running somewhere.

DuckDB does **not**. It runs *inside* the program you are already using — a Python script, an R session, a BI tool, or a web browser.

> *Nothing to install on a server. Nothing to configure.*

```
import duckdb
con = duckdb.connect()          # That's it. You have a database.
con.sql("SELECT 42 AS answer").show()
```

"SQLite for Analytics"

People sometimes call DuckDB **"SQLite for Analytics."**

- **SQLite** is small and simple. Great for basic tasks, like saving an app's settings — this is called a **transactional** workload.
- **DuckDB** is also small and simple, but built for a different job: asking big questions over lots of data — this is called an **analytical** workload.

Created by Mark Raasveldt and Hannes Mühleisen at CWI Amsterdam.
First released in 2019; production-stable since 2024.

The Problem DuckDB Solves

Traditional analytics workflows have **friction at every step**:

Step	Traditional Way	Pain Point
Get data	Export CSV from database	Slow, extra step
Load data	Write ETL scripts	Boilerplate code
Analyze	Pandas / R / SQL Server	Memory limits, speed
Share results	Set up a database server	Ops overhead

DuckDB eliminates the middle steps.

You point it at your CSV, Parquet, or JSON file and run SQL directly — no loading, no ETL, no server.

```
duckdb.sql("SELECT * FROM read_csv('sales_2024.csv') WHERE revenue > 10000").show()
```

Core Architecture: Why It's Fast

DuckDB uses a **columnar-vectorized execution engine** — the same design behind enterprise data warehouses, but embedded.

Columnar storage: Data is organized by column, not by row. Analytical queries that scan one or two columns out of fifty skip everything else.

Vectorized execution: Instead of processing one row at a time, DuckDB processes batches of thousands of values in tight CPU loops, exploiting modern hardware caches.

Parallel processing: Queries automatically use all your CPU cores. A 4-core laptop runs queries 4× faster with zero configuration.

Zero-copy integration: When you query a Pandas DataFrame, DuckDB reads it in place — no copying the data.

The result: queries that take minutes in Pandas often finish in seconds in DuckDB.

When Should You Use DuckDB?

DuckDB shines when you need to:

- Analyze CSV, Parquet, or JSON files without setting up a database
- Run SQL queries against Pandas/Polars DataFrames
- Process datasets that are too large for Pandas but don't need a cluster
- Build data pipelines that run on a single machine
- Prototype analytical queries before deploying to a warehouse
- Embed a database inside a desktop or mobile application, or even a web browser (via WebAssembly)

DuckDB is probably not the right choice for:

- High-concurrency web applications (many users writing at once)
- Real-time transactional workloads (use PostgreSQL or SQLite)

DuckDB vs. the Alternatives

Feature	DuckDB	SQLite	PostgreSQL	Pandas
Setup needed	None	None	Server install	None
Best for	Analytics	Transactions	Both	Small data
Handles 10 GB+	Yes	Slowly	Yes	Crashes
SQL support	Full	Full	Full	No
Columnar engine	Yes	No	No	N/A
Parallel queries	Yes	No	Yes	Limited
Read CSV/Parquet	Built-in	No	Extensions	Yes
In-process	Yes	Yes	No	Yes

DuckDB fills the gap between "quick scripting" tools like Pandas and "heavy infrastructure" databases like PostgreSQL.

Superpower: Query Anything with SQL

DuckDB reads multiple formats **directly in SQL** — no import step.

```sql
-- CSV files
SELECT * FROM read_csv('customers.csv');
-- Parquet files (columnar, compressed)
SELECT * FROM read_parquet('events/*.parquet');
-- JSON files
SELECT * FROM read_json('api_response.json');
-- Pandas DataFrames (just use the variable name!)
SELECT * FROM my_dataframe WHERE status = 'active';
-- Even remote files over HTTP
SELECT * FROM read_parquet('https://example.com/data.parquet');
```

You can also **join across formats** in one query — CSV, Parquet, DataFrame together.

Example: Your First Table

No server, no setup — just create a table and insert some rows.

```
CREATE TABLE students (
    id    INTEGER,
    name  VARCHAR,
    age   INTEGER,
    grade DOUBLE
);

INSERT INTO students VALUES
    (1, 'Alice',   20, 3.8),
    (2, 'Bob',     22, 3.5),
    (3, 'Charlie', 21, 3.9),
    (4, 'Diana',   23, 3.2);
```

That's it — `students` now lives in memory, ready to query.

Example: Filter and Sort

```
SELECT name, grade
FROM students
WHERE grade > 3.6
ORDER BY grade DESC;
```

```
┌─────────┬─────────┐
│  name   │ grade   │
├─────────┼─────────┤
│ Charlie │  3.9    │
│ Alice   │  3.8    │
└─────────┴─────────┘
```

`WHERE` filters rows; `ORDER BY` sorts them. Standard SQL — nothing DuckDB-specific.

Example: Aggregates and Grouping

```
SELECT
    age,
    COUNT(*)              AS num_students,
    ROUND(AVG(grade), 2) AS avg_gpa
FROM students
GROUP BY age
ORDER BY age;
```

DuckDB supports every standard aggregate — `COUNT`, `SUM`, `AVG`, `MIN`, `MAX` — and combines them with `GROUP BY` to summarize data by category, just like a pivot table.

Example: Query a CSV — No Loading Step

Given a file `cities.csv` with columns `city, country, population`:

```sql
SELECT
    country,
    COUNT(*)              AS num_cities,
    SUM(population)        AS total_pop
FROM read_csv('cities.csv')
GROUP BY country
ORDER BY total_pop DESC;
```

No `CREATE TABLE`, no `COPY`, no import wizard. The file **is** the table.

Try It Yourself

These examples (and more) are ready to run in:

`duckdb_intro_notebook.py` — a Marimo notebook in this same folder.

It walks through, step by step:

- Creating tables and inserting data
- `SELECT`, `WHERE`, `ORDER BY`, `GROUP BY`
- Using Python variables inside SQL
- Querying a Pandas DataFrame and a CSV file with SQL
- Getting results back as a DataFrame with `.df()`

Open it with `marimo edit duckdb_intro_notebook.py` and run each cell.

Real-World Example: Log Analysis

Imagine you have 2 GB of web server logs in CSV format.

```
import duckdb
# One line to answer: "Which pages got the most 500 errors yesterday?"
duckdb.sql("""
    SELECT
        url,
        COUNT(*)            AS error_count,
        COUNT(DISTINCT ip)  AS unique_users
    FROM read_csv('access_log_*.csv')
    WHERE status_code = 500
      AND timestamp >= CURRENT_DATE - INTERVAL 1 DAY
    GROUP BY url
    ORDER BY error_count DESC
    LIMIT 10
""").show()
```

This runs in seconds on a laptop — no loading, no memory limits.

Getting Started in 60 Seconds

Install:

```
pip install "duckdb>=2.0"
```

In-memory (ephemeral):

```
import duckdb
con = duckdb.connect()            # data lives in RAM, gone when script ends
```

Persistent (saved to disk):

```
con = duckdb.connect('my_data.db')  # data survives across sessions
```

Quick one-liner (no connection needed):

```
duckdb.sql("SELECT * FROM read_csv('data.csv') LIMIT 5").show()
```

Works everywhere: Python, R, Java, Node.js, Rust, Go, C/C++, and the browser (WASM).

Start Using DuckDB Today

A fast, simple database that lives inside your own program —
no server, no setup, just answers.

Replace your next `pd.read_csv()` + filtering + groupby
with a single `duckdb.sql(...)` call — and feel the difference.

Resources

duckdb.org — Official docs and guides
github.com/duckdb/duckdb — Source code
duckdb.org/docs/guides — Tutorials and recipes

Install now:

```
pip install "duckdb>=2.0"
```